

# Gradient Descent vs. Genetic Algorithm for Urban Sound Classification

Custom Smart Adaptive Systems · UPC 2025/26

Leonie Greber

June 17, 2026

## Outline

Overview

Dataset

Architecture

Gradient Descent

Genetic Algorithm

Parameter Tuning

Results

## Motivation

### Why urban sound classification?

- ▶ Noise-level monitoring in smart cities
- ▶ Automatic detection of emergency vehicles (sirens)
- ▶ Public safety systems (gunshot detection)
- ▶ Traffic and construction management

**This project:** train the *same* MLP two fundamentally different ways:

- **GD** (Adam): follow  $\nabla \mathcal{L}$ , *gradient-guided*
- **GA** (DEAP): evolve weight vectors, *gradient-free*

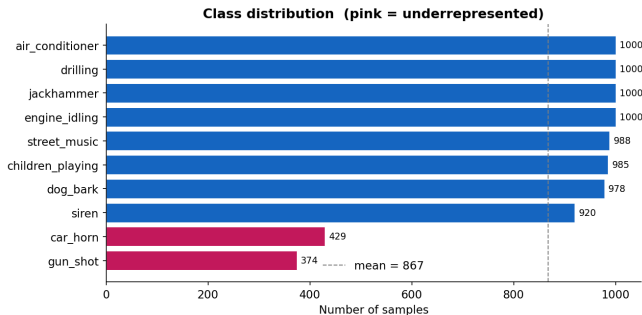
### Dataset: UrbanSound8K

|          |                                   |
|----------|-----------------------------------|
| Clips    | 8 674 audio samples               |
| Classes  | 10 urban sound types              |
| Features | 34 per clip (MFCC, Chroma, ...)   |
| Split    | 70 % train / 10 % val / 20 % test |

### Research question

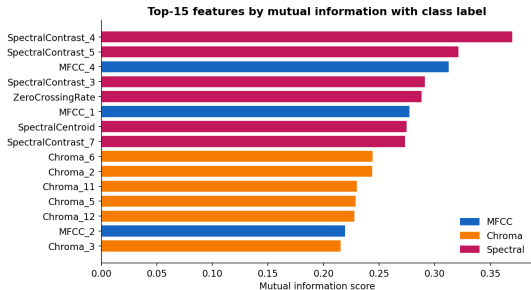
Which optimizer finds better weights for the same MLP: gradient-guided Adam or gradient-free evolution?

## Class Distribution

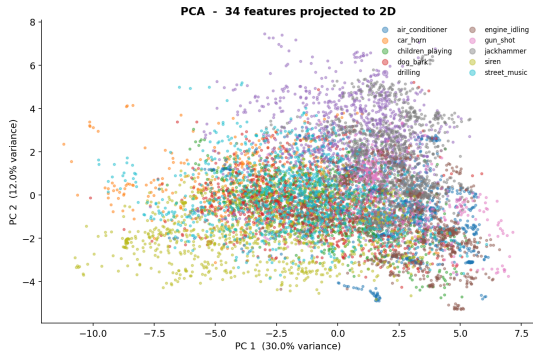


- ▶ Most classes:  $\approx 1\,000$  samples
- ▶ **car\_horn** ( $\approx 429$ ) and **gun\_shot** ( $\approx 374$ ) are underrepresented
- ▶ Stratified split keeps proportions in every fold
- ▶ These two classes show lower F1 (by design, not model failure)

## Feature Analysis



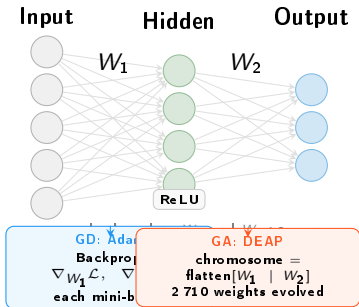
MFCC features dominate mutual information



PC1+PC2 explain  $\approx 42\%$  of variance; classes overlap

- ▶ Heavy class overlap even in the most informative 2D projection: a **non-linear model is needed**
- ▶ MFCC features capture the spectral shape that makes each sound type unique

## MLP Architecture



### Forward pass:

$$h = \text{ReLU}(W_1 x + b_1), \quad z = W_2 h + b_2$$

| Layer         | Formula             | Params       |
|---------------|---------------------|--------------|
| Linear(34→60) | $34 \times 60 + 60$ | 2 100        |
| Linear(60→10) | $60 \times 10 + 10$ | 610          |
| <b>Total</b>  |                     | <b>2 710</b> |

- ▶ Same architecture for both; only the weight-update mechanism differs
- ▶ GA encodes all 2 710 weights as a flat chromosome
- ▶ hidden\_dim = 60 chosen by Optuna (Section 4)

## Gradient Descent

**Cross-entropy loss** (multi-class,  $N$  samples,  $C = 10$  classes):

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{y_i}(x_i; \theta) \quad (1)$$

$$\hat{p}_k(x) = \frac{e^{z_k(x)}}{\sum_{j=1}^C e^{z_j(x)}} \quad (\text{softmax}) \quad (2)$$

**Vanilla gradient descent update:**

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (3)$$

- ▶  $\nabla_{\theta} \mathcal{L}$  computed via **backpropagation** (chain rule through both linear layers)
- ▶ Mini-batch size 64: trade-off between gradient noise and convergence speed
- ▶ With weight decay  $\lambda$ : effective loss is  $\mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|^2$

## The Adam Optimizer

Adam maintains *per-parameter* running estimates of gradient mean and variance:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (5)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction}) \quad (6)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (7)$$

### Tuned parameters:

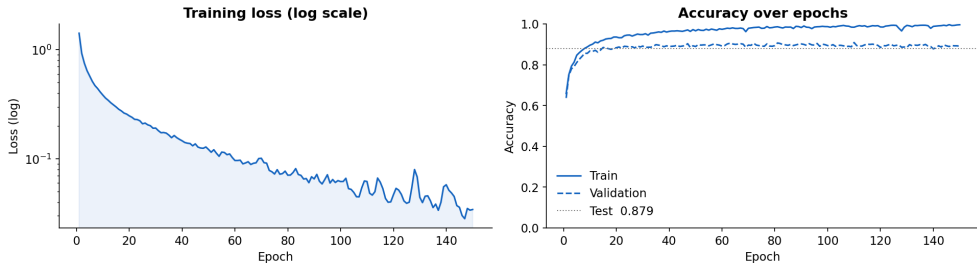
$$\eta = 4.96 \times 10^{-3}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999$$
$$\epsilon = 10^{-8}, \quad \lambda_{\text{wd}} = 3.29 \times 10^{-5}$$

- ▶  $\sqrt{\hat{v}_t}$  normalises the step: large-gradient params take smaller steps
- ▶ Bias correction avoids early steps being too small when  $m_0 = v_0 = 0$



## GD Training Curves

Gradient Descent (Adam)



- ▶ Loss (log scale) falls near-linearly, plateauing after  $\approx 80$  epochs
- ▶ Train and val accuracy track closely: no overfitting despite 2 710 parameters
- ▶ Converges in 150 epochs; total wall time  $\approx 52$  s

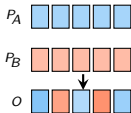
## GA Operators

### Tournament Selection



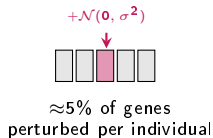
Draw 5 at random;  
highest fitness wins

### BLX- $\alpha$ Crossover



Blend genes from  
both parents

### Gaussian Mutation



### Fitness signal:

$$f(\theta) = -\mathcal{L}_{\text{val}}(\theta)$$

Continuous loss gives far more selection pressure than coarse accuracy.

### Elitism:

Top  $k = 1$  individual copied unchanged each generation. The best solution never regresses.

### Why GA is hard here:

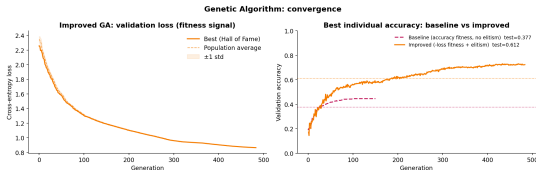
Searching 2710 dimensions without gradient information is like finding the lowest point of a mountain range blindfolded.

## GA Convergence

### One generation

1. Evaluate  $f(\theta) = -\mathcal{L}_{\text{val}}$  for all invalid individuals
2. Select  $N - k$  offspring via tournament (size 5)
3. BLX- $\alpha$  crossover with probability  $c = 0.719$
4. Gaussian mutation with probability  $p_m = 0.045$
5. Carry over top  $k = 1$  elite individual unchanged
6. Update Hall of Fame (best ever seen)

- ▶ **Baseline** (accuracy fitness, no elitism): test acc 0.377
- ▶ **Improved** ( $-\mathcal{L} + \text{elitism}$ ): test acc 0.612, early stopping at gen 483



- ▶ Left: val loss of HoF vs population average; gap widens as elites dominate
- ▶ Right: improved GA far outperforms baseline

## Hyperparameter Tuning

### Optuna (Tree-structured Parzen Estimator)

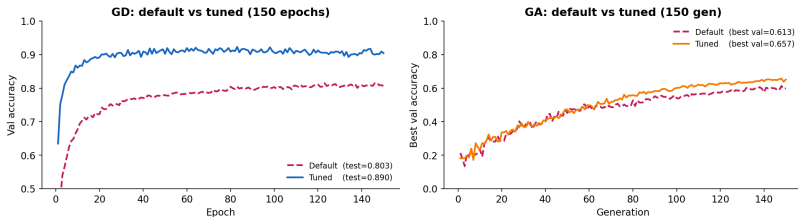
1. After each trial, fit two densities over the search space:  
 $\ell(x)$ : distribution of *good* configurations  
 $g(x)$ : distribution of *bad* configurations
2. Propose next trial by maximising  $\ell(x)/g(x)$
3. **MedianPruner**: stop a trial early if it falls below the median of completed trials at the same step

More sample-efficient than grid or random search.

| Parameter                            | Range                | Type    |
|--------------------------------------|----------------------|---------|
| <i>GD: 60 trials, 60 epochs each</i> |                      |         |
| lr                                   | $[10^{-4}, 10^{-2}]$ | log     |
| weight_decay                         | $[10^{-6}, 10^{-2}]$ | log     |
| hidden_dim                           | {45, 50, 55, 60, 65} | cat.    |
| <i>GA: 20 trials, 100 gen each</i>   |                      |         |
| pop_size                             | {50, 100, 150, 200}  | cat.    |
| cxbp                                 | [0.5, 0.9]           | uniform |
| mutpb                                | [0.01, 0.20]         | uniform |
| elite_k                              | {1, 3, 5, 8, 10}     | cat.    |

## Tuning Results

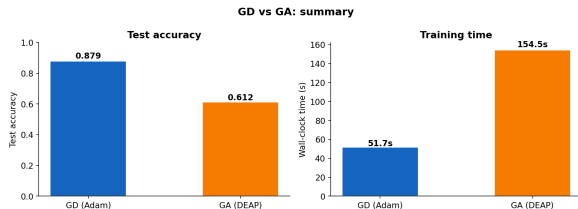
Tuning validation: default vs Optuna-tuned hyperparameters



- **GD:** val acc rises from  $\approx 83\%$  (default  $h = 20$ ,  $\eta = 10^{-3}$ ) to  $91.9\%$  (tuned  $h = 60$ ,  $\eta = 4.96 \times 10^{-3}$ )

- **GA:** modest improvement; 20 trials insufficient for a reliable 4-parameter search
- Tuning pays off strongly for GD; gradient-based methods are simply more tunable

## Accuracy and Training Time

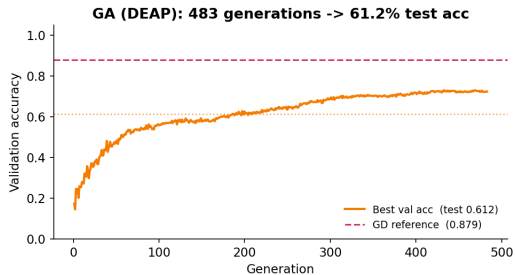
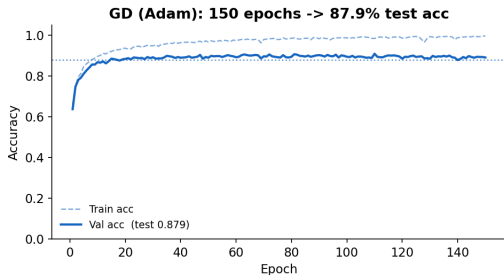


|               | GD             | GA              |
|---------------|----------------|-----------------|
| Test accuracy | <b>0.879</b>   | 0.612           |
| Training time | $\approx 52$ s | $\approx 155$ s |
| Macro F1      | <b>0.879</b>   | 0.614           |
| Stopped at    | 150 epochs     | gen 483         |

- ▶ GD is  $\approx 27\%$  more accurate and 3 $\times$  faster
- ▶ GA struggles in 2 710 dimensions without gradient information
- ▶ Elitism and  $-\mathcal{L}$  fitness doubled GA accuracy over the baseline ( $0.377 \rightarrow 0.612$ )

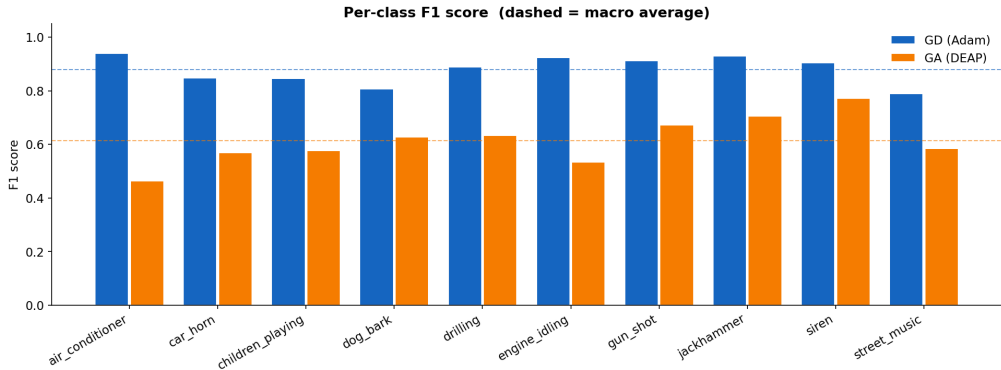
## Convergence

Convergence: GD vs GA



- ▶ **Left:** GD val accuracy on its own epoch axis; converges in <80 epochs
- ▶ Train and val curves track closely: no overfitting
- ▶ **Right:** GA best val accuracy on its own generation axis; pink dashed = GD reference
- ▶ 483 generations and 3× more wall time to still fall 27% short

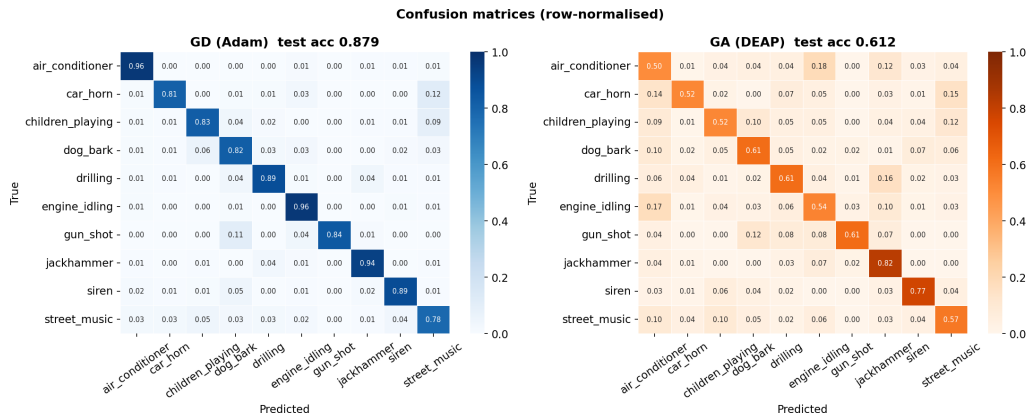
## Per-class F1 Score



- ▶ GD dominates on all 10 classes; both methods struggle most with *gun\_shot* and *car\_horn*
- ▶ Low F1 on those two classes reflects the class imbalance, not model failure



## Confusion Matrices



- ▶ GA confusion matrix shows far more off-diagonal mass; less discriminative across all classes
- ▶ GD confuses acoustically similar pairs (e.g. jackhammer/drilling); GA confuses almost everything

## Conclusion

### Key findings:

- ▶ GD (Adam) clearly wins: 87.9% vs 61.2% accuracy,  $3\times$  faster
- ▶ Optuna tuning improved GD from 81% to 88%; GA gains were modest, confirming GD is the more tractable target
- ▶ Continuous  $-\mathcal{L}$  fitness and elitism are *essential* for GA (baseline: 37.7%)
- ▶ Both methods agree: *gun\_shot* and *car\_horn* are hardest

### When does GA make sense?

- ▶ Non-differentiable objectives (hardware-in-the-loop, discrete search spaces)
- ▶ Black-box optimisation where gradient computation is impossible

### Future work:

- ▶ CMA-ES: adapts mutation covariance, far more efficient in high dimensions
- ▶ Hybrid: GD warm-start, then GA fine-tune for non-smooth surfaces
- ▶ Neuroevolution (NEAT): evolve topology, not just weights